

Republic of Yemen
Ministry of Higher Education & Scientific Research
Taizz University
Alsaeed Faculty of Engineering & IT
IT Department

LECTURE [01]

Fundamentals of Systems, Architecture & Integration

Topics: Fundamentals of Systems & Architecture, System Integration, Heterogeneous Systems & Complexity



IT Department — 4th Level



LECTURER

Eng/ Salah Alssayani

ACADEMIC YEAR 2024 — 2025

Fundamentals of Systems, Architecture & Integration

An introduction and overview of the core course — system fundamentals, integration principles, and heterogeneous environments.

Lecture: Lecture 1 of the System Architecture Course

Topics: Fundamentals of Systems & Architecture, System Integration, Heterogeneous Systems & Complexity

Lecturer: Eng. Salah Alssayani



Lecture Roadmap — Three Core Topics

This lecture introduces the foundational concepts of system architecture and integration. We cover three core topics: (1) Fundamentals of Systems & Architecture — what systems are, core principles, system vs. enterprise architecture, and architectural and architectural views. (2) Fundamentals of System Integration — concept, importance, integration vs. migration, and coupling and coupling strategies. (3) Heterogeneous Systems & Complexity — diverse environments, interoperability challenges, and cross-challenges, and cross-system governance.

01 Fundamentals of Systems & Architecture



Concept & definition of a system. Core architecture principles. System vs. enterprise architecture. Architectural views, stakeholders, & concerns.

02 Fundamentals of System Integration



Concept & scope of integration. Strategic & operational importance. Integration vs. migration. Direct coupling vs. decoupled integration.

03 Heterogeneous Systems & Complexity



Nature of heterogeneous environments. Diversity in hardware, OS, data models, protocols. Interoperability, legacy, siloed architectures. Data consistency, security, & governance.

◆ Course foundation

This lecture establishes the vocabulary and mental models for the entire System Architecture & Integration course. Every subsequent lecture builds on these foundational concepts.

01 / 03

Fundamentals of Systems & Architecture

What is a system? What is architecture? How do architectural views shape our understanding of complex software?

SYSTEM DEFINITION

CORE PRINCIPLES

VIEWS & STAKEHOLDERS

"Architecture is the set of decisions whose cost-of-change grows over time."

Concept & Definition of a System

A system is a collection of components that work together to achieve a common purpose. In computing, a system includes hardware, software, data, people, and processes — all coordinated to deliver a specific function. The study of systems theory (von Bertalanffy, 1968) established that the whole is greater than the sum of its parts — emergent properties arise from component interactions that no single component possesses alone.

Components

Hardware, software, data, people, processes. Each has a defined role.

Interactions

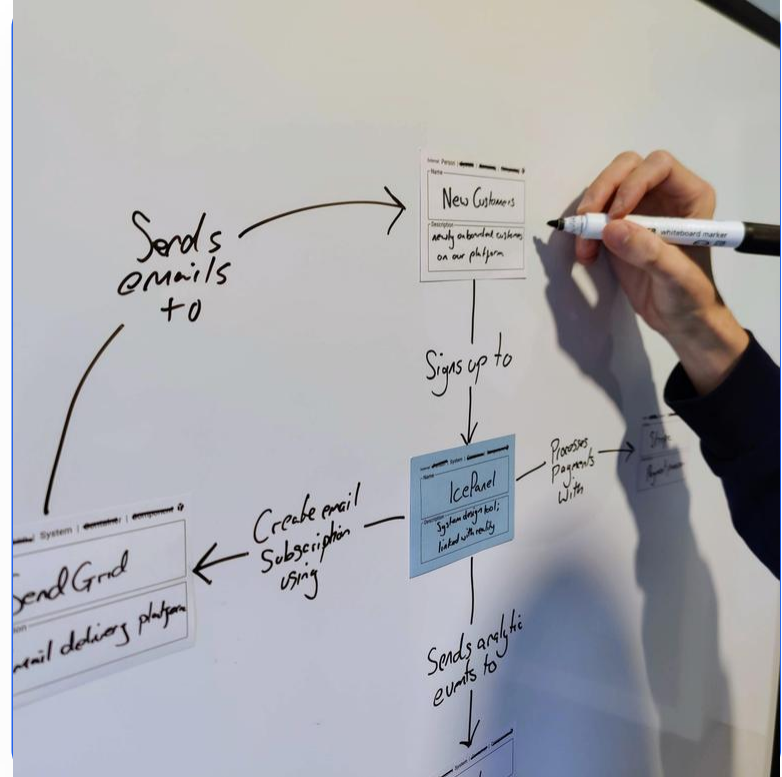
Components communicate via interfaces. Interactions produce emergent behavior.

Purpose

Every system has a goal. Without purpose, it is just a collection of parts.

Boundary

Systems have boundaries — what is inside (the system) vs. outside (the environment).



A system is more than its parts — architecture defines how they interact.

System Architecture Core Principles

Six foundational principles that govern every architectural decision.

Separation of Concerns

Each module addresses one concern. Dijkstra (1974). Reduces complexity; enables independent evolution.

Trade-off Analysis

Every architecture decision optimizes for some quality attribute at the cost of another — latency vs. cost, consistency vs. availability, security vs. convenience. The architect job is to make trade-offs explicit and defensible. BCK ATAM (Architecture Trade-off Analysis Method) provides a structured framework for surfacing these trade-offs with stakeholders.

Abstraction

Hide complexity behind simple interfaces. Consumers see the contract, not contract, not the implementation.

Loose Coupling

Minimize dependencies between components. Changes in one do not ripple to others.

High Cohesion

All code in a module serves one purpose. Related functionality stays together.

Principle of Least Privilege

Grant minimum access needed. Applies to security, security, modules, and service boundaries.

System vs. Enterprise Architecture

Three levels of architectural scope — each with different concerns and stakeholders.

Dimension	System Architecture	Enterprise Architecture (EA)	Software Architecture
Scope	One system end-to-end (hardware + software + network)	Whole organization (business + IT + data + processes)	Software only (components, modules, services)
Time horizon	Years (system lifetime)	Years to decades (org lifetime)	Months to years (release cycles)
Stakeholders	System owners, ops, users	C-suite, business units, all IT	Developers, architects, product owners
Concerns	Hardware, deployment, network, HA	Business capabilities, data domains, vendor strategy	Code structure, patterns, frameworks
Frameworks	BCK, Richards & Ford	TOGAF, Zachman, ArchiMate	GoF, SOLID, DDD, EIP
Examples	The order system runs on K8s in 3 regions	All customer-facing apps use SSO via Okta	OrderService has PricingService + InventoryAdapter

System Arch

Bridges software and infrastructure. Includes hardware, network, deployment topology.

Enterprise Arch

Highest level. Aligns IT with business strategy. TOGAF, Zachman frameworks.

Software Arch

Narrowest. Code-level decisions — patterns, frameworks, module boundaries.

Architectural Views, Stakeholders & Concerns

No single diagram can represent a complex system. BCK identify three identify three structure categories: Module (code organization), organization), Component-and-Connector (runtime behavior), and and Allocation (deployment mapping). Each view answers different different questions for different stakeholders. Kruchten 4+1 View Model View Model adds Logical, Process, Development, and Physical views plus Scenarios. The architect must choose which view serves serves which conversation.

◆ Module View

How is code organized? Packages, layers, components. For developers.

↔ C&C View

How does it run? Services, processes, queues. For ops/SREs.

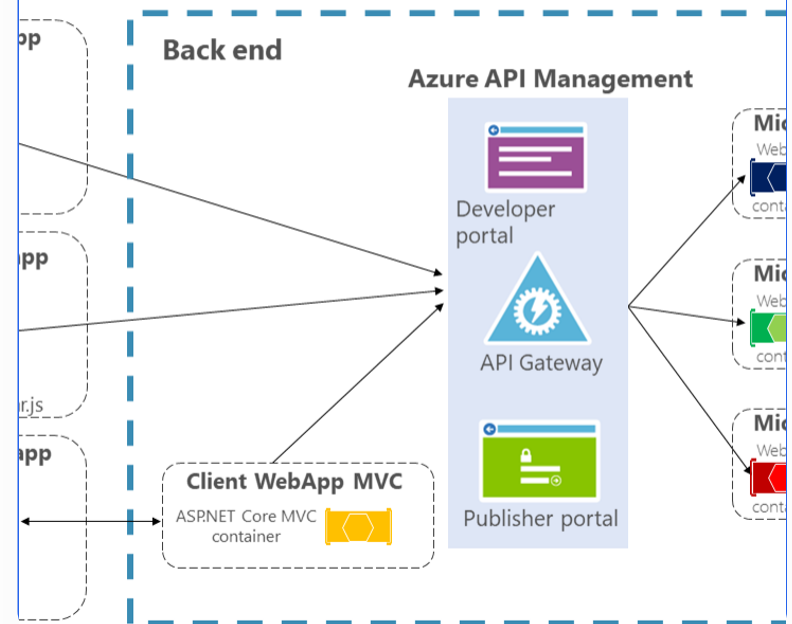
☁ Allocation View

Where does it live? Servers, regions, teams. For devops/finance.

👤 Stakeholder Map

Each view maps to different stakeholders — missing a view means missing a conversation.

API Gateway with Azure API Management Architecture



Multiple views of the same system — each serving different stakeholders.

Fundamentals of System Integration

What integration is, why it matters, how it differs from migration, and the coupling strategies that govern every integration project.

CONCEPT & SCOPE

IMPORTANCE

COUPLING STRATEGIES

*Integration is harder than building — you inherit every decision made by people who left.
who left.*

Concept & Scope of System Integration

System integration is the discipline of connecting independent systems — each with its own data model, protocols, and lifecycle — so they cooperate as if they were one coherent system. Linthicum (1999) defined it as bringing together component subsystems into a whole. The scope spans data-level (ETL, CDC), application-level (APIs, messaging), process-level (BPM, workflows), and UI-level (portals, SSO) integration.

Data-Level

Shared databases, file transfer, transfer, ETL pipelines. Moving bytes between systems.

Application-Level

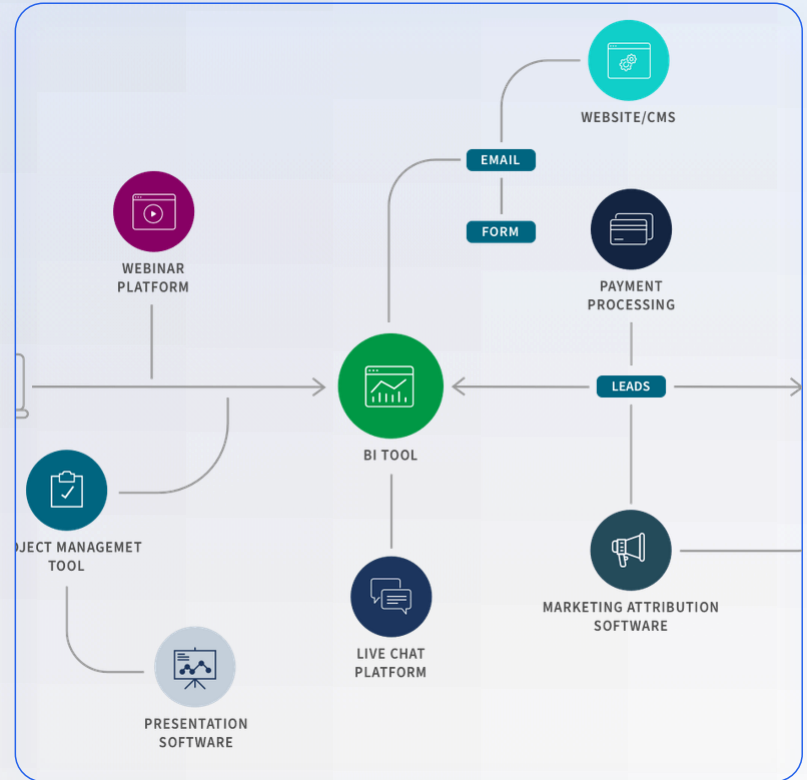
APIs, RPC, messaging brokers. Coordinating behavior across services.

Process-Level

BPM, saga orchestration. Coordinating business workflows.

UI-Level

Portals, mashups, SSO. Unifying the user experience.



A typical integration topology — connecting SaaS, on-prem, and cloud systems.

Strategic & Operational Importance of Integration

Why integration is not just a technical concern — it is a business enabler.

Cost Reduction

Eliminate manual data re-entry across systems. One insurance company cut 40% of of back-office staff after EAI.

Single Source of Truth

Consolidate data across Salesforce, Workday, Workday, ServiceNow for Customer 360 view. Enables real-time dashboards, reporting, reporting, and analytics. Without integration, integration, each SaaS app is a data silo — the the very problem cloud was supposed to solve. solve. Integration is the bridge from data to insight.

Faster Processes

Real-time order-to-cash instead of instead of nightly batches. Cycle Cycle time from days to minutes. minutes.

Agility

Swap Salesforce for HubSpot without rewriting SAP integration. integration. Decoupling enables enables change.

M&A Integration

Acquired company systems must interoperate. interoperate. Integration patterns enable faster faster post-merger integration.

Compliance

GDPR, SOX, HIPAA require end-to-end-to-end data lineage. Integration layer provides the audit trail.

System Integration vs. System Migration

Two related but distinct disciplines — confusing them causes project failures.

INTEGRATION



- **Goal:** Make independent systems cooperate as one
- **Systems stay in place** — they are connected, not replaced
- **Risk:** Data inconsistency, coupling, performance
- **Approach:** APIs, messaging, ETL, ESB, iPaaS
- **Timeline:** Continuous — integration is never done
- **Identity:** Systems retain their identity and ownership
- **Example:** Connect Salesforce CRM to SAP ERP for real-time order sync
- **Key metric:** End-to-end process latency

Cooperate

MIGRATION



- **Goal:** Move from old system to new system
- **Old system is retired** — new system replaces it
- **Risk:** Data loss, downtime, user disruption
- **Approach:** Data conversion, parallel runs, cutover
- **Timeline:** Finite project — ends at cutover
- **Identity:** Old system loses identity — replaced entirely
- **Example:** Move from on-prem Oracle to cloud Snowflake
- **Key metric:** Cutover success rate, data integrity

Replace

◆ Key Distinction

Integration connects systems that coexist. Migration replaces one system with another. Integration is continuous; migration is a project. Many failures occur when teams treat integration as a one-time migration project — it never ends.

Direct Coupling vs. Decoupled Integration

The fundamental architectural choice — tight and synchronous, or loose and asynchronous?

Dimension	Direct (Tight) Coupling	Decoupled (Loose) Integration
Communication	Synchronous request-response (REST, gRPC)	Asynchronous events (Kafka, RabbitMQ, pub/sub)
Coupling	Tight — caller knows callee	Loose — producer does not know consumer
Failure mode	Caller fails if callee down (cascading)	Broker buffers; consumer catches up
Latency	Low (sub-ms to seconds)	Higher (seconds to minutes)
Consistency	Strong (immediate)	Eventual (converges over time)
Scalability	Limited by callee capacity	High — add consumers freely
Debugging	Easy — single call stack	Hard — distributed tracing required
Best for	Reads, queries, RPC	Writes, side effects, fan-out, audit

✓ Choose Direct when

You need an answer to continue. Read paths, validation, RPC commands. Caller commands. Caller cannot proceed without response.

✓ Choose Decoupled when

You can return success before downstream completes. Fire-and-forget, side forget, side effects, fan-out notifications, audit.

Heterogeneous Systems & Complexity

Diverse environments, interoperability challenges, and cross-system governance.

HETEROGENEOUS NATURE

DIVERSITY & CHALLENGES

CONSISTENCY, SECURITY & GOVERNANCE

GOVERNANCE

The real world is heterogeneous — different hardware, OS, data models, and protocols must cooperate.

Nature & Characteristics of Heterogeneous Environments

A heterogeneous environment is one where systems differ in hardware, operating systems, programming languages, data models, network protocols, and ownership. This is the default state of enterprise IT — not a problem to fix, but a reality to manage. Heterogeneity arises from decades of independent purchasing decisions, mergers and acquisitions, technology evolution, and the persistence of legacy systems that cannot be replaced.

◆ Historical Accumulation

Decades of independent decisions. Each team bought what was best at the time.

◆ M&A Legacy

Acquired companies bring their own stack. Integration takes years.

◆ Technology Evolution

New systems use REST/JSON; old use SOAP/XML. Both must coexist.

🔑 Legacy Persistence

Mainframes, COBOL, AS/400 still run critical critical processes. Cannot be replaced.

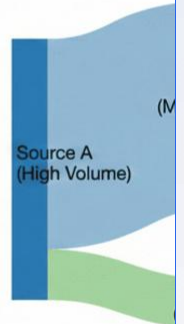
Diagram: Relationships



Flowchart: Shows a Process



Sankey Shows



Heterogeneous environments require multiple integration approaches.

Diversity in Hardware, OS, Data Models & Network Protocols

Four dimensions of heterogeneity that the integration architect must bridge.

❑ Hardware Diversity

x86 servers, ARM, mainframes (IBM Z), GPUs (NVIDIA), IoT devices. Different instruction sets, sets, memory models, and I/O capabilities.

🗃️ Data Model Diversity HARDEST TO BRIDGE

Relational (PostgreSQL, Oracle) vs. document (MongoDB) vs. key-value (Redis) vs. graph (Neo4j) vs. columnar (Cassandra) vs. time-series series (InfluxDB). Each optimized for different access

● OS & Runtime Diversity

Linux (Ubuntu, RHEL, Alpine), Windows Server, AIX, z/OS. JVM, .NET CLR, Node.js, Python, Go. Different file systems, processes, libraries.

● Network Protocol Diversity

HTTP/REST, gRPC, SOAP, JMS, AMQP, MQTT, proprietary TCP. Different encoding, security, and reliability models.

● Language Diversity

Java, C#, Python, Go, JavaScript, COBOL, Rust. Different type systems, memory models, and concurrency primitives.

☁️ Deployment Diversity

On-prem VMs, Kubernetes, serverless (Lambda), (Lambda), containers (Docker), cloud-managed (RDS, managed (RDS, BigQuery). Different scaling and ops and ops models.

Challenges in Heterogeneous Systems

Interoperability, legacy systems, and siloed architectures — the three core challenges.

Challenge	Description	Impact	Mitigation
Interoperability	Systems cannot communicate due to different protocols, data formats, and APIs	Manual data re-entry; delayed processes; errors	Protocol bridges (ESB, API gateway); canonical data models; adapters
Legacy Systems	Mainframes, COBOL, AS/400 — critical but inflexible	Cannot expose REST APIs; hard to integrate; vendor lock-in	Anti-corruption layer; screen scraping; progressive modernization
Siloed Architectures	Each department has its own system with no integration	Data duplication; inconsistent reports; no single source of truth	Integration platform (iPaaS); master data management; API-led connectivity
Data Inconsistency	Same entity has different data in different systems	Customer 360 impossible; compliance violations	Master data management; event-driven sync; conflict resolution
Vendor Lock-in	Proprietary APIs, data formats, and processes	Cannot switch vendors; high switching cost	Open standards (REST, OpenAPI); abstraction layers; anti-corruption layer
Skills Gap	Different teams know different technologies	Integration projects require multi-disciplinary teams	Cross-training; iPaaS with low-code; center of excellence

Data Consistency, Security & Governance across Non-Homogeneous Systems

Three cross-cutting concerns that span all heterogeneous systems.

Data Consistency

Same data exists in multiple systems with different different values. ACID (strong) vs. BASE (eventual). (eventual). Distributed transactions (2PC) vs. Sagas. Sagas. Conflict resolution: LWW, CRDTs, application-application-level.

Data Governance

Who owns the data? Who can access it? Data catalog (Collibra, Alation). Data lineage: where did this field come from? Master data management (MDM).

Security & Compliance #1 HYBRID CHALLENGE

Each system has its own auth model (LDAP, AD, OAuth2). Data in transit: TLS 1.3, mTLS. Data at rest: AES-256. Compliance: GDPR (data residency), HIPAA (health data), SOC 2 (audit). Secrets management:

Observability

Centralized logging, metrics, distributed tracing. tracing. OpenTelemetry standard. Cross-system system audit trail for compliance.

Authentication

SSO (SAML, OIDC) federates identity across systems. MFA mandatory. Zero Trust: never trust, always verify — every request authenticated.

Policy Enforcement

RBAC/ABAC across heterogeneous systems. OPA OPA (Open Policy Agent). Centralized policy, distributed enforcement.

Integration Patterns & Tools — Overview

A quick reference for the tools and patterns covered in this course.

DATA

DATA-LEVEL INTEGRATION

ETL (Informatica, dbt, Fivetran) • CDC (Debezium, (Debezium, AWS DMS) • File transfer (SFTP, S3) • S3) • Schema mapping (Altova, JOLT). For batch and batch and near-real-time data sync.

APP

APPLICATION-LEVEL INTEGRATION

APIs (REST, gRPC, GraphQL) • Messaging (Kafka, RabbitMQ, SQS/SNS) • EIP patterns (65, Hohpe & Woolf) • ESB (Apache Camel, MuleSoft). For service-to-service communication.

CLOUD

CLOUD & HYBRID INTEGRATION

iPaaS (MuleSoft, Boomi, Workato, Azure Logic Apps) Logic Apps) • API Gateway (Kong, AWS API GW, GW, Apigee) • Hybrid connectivity (VPN, Direct Direct Connect). For cloud-native and hybrid

Security Tools

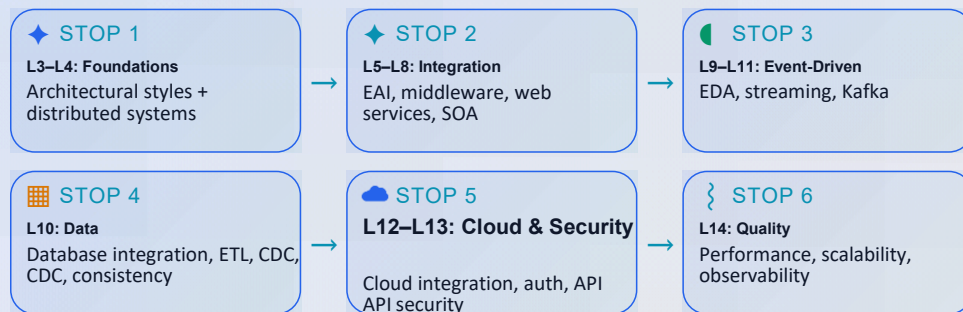
- Identity: Okta, Auth0, Azure AD, Keycloak
- Auth: OAuth2, OIDC, SAML, JWT
- Encryption: TLS 1.3, AES-256, mTLS (Istio)
- Secrets: HashiCorp Vault, AWS KMS
- API Security: rate limiting, WAF, CORS

Governance & Observability

- Data Catalog: Collibra, Alation, DataHub
- Master Data: MDM platforms
- Policy: OPA (Open Policy Agent)
- Monitoring: Prometheus, Grafana, Datadog
- Tracing: OpenTelemetry, Jaeger, Tempo
- Compliance: GDPR, HIPAA, SOC 2, ISO 27001

Course Roadmap — What Comes Next

This lecture established the foundations. The course continues with 11 more lectures covering foundational architectural styles, distributed systems, EAI, middleware, web services, SOA, event-driven architecture, database integration, cloud integration, security, and quality attributes. Each lecture builds on the vocabulary and concepts introduced here.



Recommended First Reading



BOOK Ch.1

Bass, Clements, Kazman — Software Architecture in Practice, 4/e (2021).



Lecture Ch.1-2

Enterprise Application Integration (1999).



Knapman Ch.1

Designing Data-Intensive Applications (2017).



Hofman & Wulf

Enterprise Integration Patterns (2003) — skim Ch.1.

Cheat Sheet — Fundamentals in One Slide

Every concept from this lecture, summarized for quick reference.

◆ Systems & Architecture

System = components + interactions + purpose. Core purpose. Core principles: separation of concerns, concerns, trade-off analysis, abstraction, loose loose coupling, high cohesion, least privilege. System vs. enterprise vs. software architecture. BCK architecture. BCK three views: Module, C&C, Allocation.

◆ System Integration

Integration = connect independent systems as one. Four levels: data, application, process, UI. Integration (continuous, systems coexist) vs. migration (finite, old system retired). Direct coupling (sync, tight) vs. decoupled (async, loose).

❄ Heterogeneous Systems

Diversity in hardware, OS, data models, protocols, protocols, languages, deployment. Challenges: Challenges: interoperability, legacy, silos. Cross-Cross-cutting: data consistency (ACID/BASE, 2PC, 2PC, Sagas), security (Zero Trust, SSO, TLS, KMS), KMS), governance (MDM, data catalog, OPA).

📖 Recommended Reading

- Bass, Clements, Kazman — Software Architecture in Practice, 4/e (2021), Ch.1–3.
- David Linthicum — Enterprise Application Integration (1999).
- Martin Kleppmann — Designing Data-Intensive Applications (2017), Ch.1.
- Gregor Hohpe, Bobby Woolf — Enterprise Integration Patterns (2003).
- Andrew Tanenbaum, Maarten van Steen — Distributed Systems, 3/e (2017).
- ISO/IEC/IEEE 42010:2022 — Architecture description.



Lecturer

Eng. Salah Alsayani

Next: Lecture 3 — Foundational Architectural Styles: Monolithic, Client-Server, Layered, N-Tier, MVC. Read BCK Ch.1 and Ch.5 before the next session.